

The image features a central teal-colored band that serves as a background for the title and author. Above and below this band are horizontal strips showing a close-up of an open notebook with lined pages and a pen resting on them.

数据结构

周宇



课程说明

课程安排及学时分配



课程安排

- 教材

- 数据结构、算法与应用----C++语言描述 Sartaj Sahni著

• 参考教材

- 数据结构（C++语言版）（第三版） 邓俊辉著
- 数据结构与算法分析-c语言描述 Mark Allen Weiss著
- 数据结构（C语言版 第2版），严蔚敏，李冬梅，吴伟民 著
- 大话数据结构 程杰 著

学时分配

- 绪论 (1*3课时)
 - 算法分析及基本概念
- 第1部分 线性数据结构 (4*3课时)
 - 顺序表, 链表, 栈, 队列等的实现分析和应用
- 第2部分 二叉树与其它树 (4*3课时)
 - 树、搜索树、平衡搜索树、B树
- 第3部分 图 (3*3课时)
 - 图的类型和定义
 - 遍历, 最小生成树, 关键路径等
- 第4部分 应用 (3*3课时)
 - 排序和查找: 基本算法分析

联系方式

- Email
 - yzhou@nankai.edu.cn
- 工作地点
 - 津南校区计算机学院556A

The background features a wide teal horizontal band across the center. Above and below this band are images of a notebook, showing its pages and a pen.

绪论

算法分析及基本概念

主要内容

- 基本概念和术语
- 算法定义
 - 算法特性
 - 算法效率度量
 - 算法复杂度渐进符号 (O 、 Ω 、 Θ 、 o)
 - 算法空间复杂度
- 性能测量方法

学习目标

- 了解数据结构的**基本概念**、**研究对象**以及数据结构课程的发展历史，对数据结构与算法课程有一个宏观的认识
- 掌握贯穿全书的重要概念-----**抽象数据类型**，包括其概念的定义和实现方法，初步掌握抽象技术方法
- 了解算法、算法复杂性，掌握算法**性能的评价方法**
- 了解解决问题的一般过程和算法的**逐步求精**方法，掌握问题 **求解的基本过程和方法**

数据结构的创始人

—Donald. E. Knuth

- 1938年出生，25岁毕业于加州理工学院数学系，博士毕业后留校任教，28岁任副教授。30岁时，加盟斯坦福大学计算机系，任教授。从31岁起，开始出版他的历史性经典巨著：

- *The Art of Computer Programming*
- 第一卷《基本算法》 1968年出版
- 第二卷《半数字化算法》 1969年出版
- 第三卷《排序与搜索》 1973年出版
- 第四卷A《组合算法》 2011年出版

计算机程序的发展

- 程序设计的实质是什么？
 - 数据表示：将数据存储在计算机中
 - 数据处理：处理数据，求解问题
 - 数据结构问题起源于程序设计
- 数据结构随着程序设计的发展而发展
 - 1. 无结构阶段：在简单数据上作复杂运算
 - 2. 结构化阶段：数据结构 + 算法 = 程序
 - 3. 面向对象阶段：（对象 + 行为） = 程序

数据结构 研究内容（Cont.）

- 求解非数值计算的问题：
 - 设计出合适的数据结构及相应的算法：
首先要考虑对相关的各种信息如何表示、组织和存储？

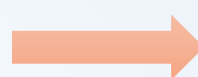
数据结构的研究内容为：

研究非数值计算的程序设计问题中计算机的**操作对象**以及它们之间的**关系和操作**。

数据结构 研究内容

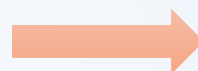
- 信息管理

- 数据检索、信息统计等

 • 链表

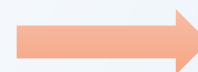
- 人机对弈

- 从不同选项中，选择前景最好的

 • 树

- 最短路径

- 选择cost最低的路径
 -

 • 图

数据结构与计算学科

Web信息处理

队列、图、字符、矩阵、哈希表、排序、索引、检索

人工智能

广义表、集合、有向图、搜索树

图形图像

队列、栈、图、矩阵、空间索引树、检索

数据库

线性表、多链表、排序、B+树

操作系统

队列、存储管理表、排序、目录树

编译原理

字符串、栈、哈希表、语法树

算法分析与设计

数据结构与算法

计算复杂性理论

程序设计语言

概率统计

计算概论

集合论与图论



数据结构定义

基本概念及术语

基本概念和术语

- 数据 (Data) :

一切能输入到计算机中并能被计算机程序识别和处理的符号集合。

- 数值数据：整数、实数等
- 非数值数据：图形、图象、声音、文字等

- 数据元素(Data Element):

数据的基本单位，在程序中通常作为一个整体进行考虑和处理 (Node, Record) 。

- 数据项(Data Item):

构成数据元素的不可分割的最小单位(Field)。

- 数据对象(Data Object):

具有相同性质的数据元素的集合，是数据的子集。

数据结构概念

- **数据结构**：数据元素及其相互间的关系的**数学描述**。数据结构是带“结构”的数据元素的集合，“结构”就是指数据元素之间存在的关系。
- 数据结构的两个层次：
 - 逻辑结构---
数据元素间抽象化的相互关系，与数据的存储无关，独立于计算机，它是从具体问题抽象出来的数学模型。
 - 存储结构（物理结构）----
数据元素及其关系在计算机存储器中的存储方式。

逻辑结构（Cont.）

- 划分方法一

- （1）线性结构----

- 有且仅有一个开始和一个终端结点，并且所有结点都最多只有一个直接前趋和一个后继。

- 例如：线性表、栈、队列、串

- （2）非线性结构----

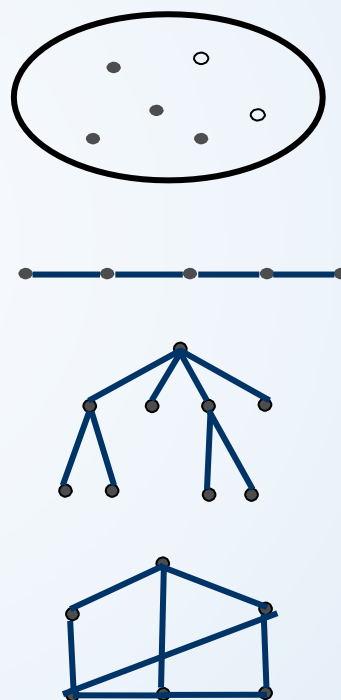
- 一个结点可能有多个直接前趋和直接后继。

- 例如：树、图

逻辑结构

• 划分方法二

- **集合**：数据元素之间就是“属于同一个集合”；
- **数据线性结构**：数据元素之间存在着一对一的线性关系；
- **树型结构**：数据元素之间存在着一对多的层次关系；
- **图结构**：数据元素之间存在着多对多的任意关系。



抽象数据类型

(ADTs: Abstract Data Types)

- 更高层次的数据抽象
- 由用户定义用以表示应用问题的数据模型
- 由基本的数据类型组成, 并包括一组相关的操作
- 抽象数据类型可以用以下的三元组来表示:

$$\text{ADT} = (\text{D}, \text{S}, \text{P})$$

数据对象 D上的关系集 D上的操作集



ADT常用定义格式

```
ADT抽象数据类型名{  
    数据对象: <数据对象的定义>  
    数据关系: <数据关系的定义>  
    基本操作: <基本操作的定义>  
} ADT抽象数据类型名
```

Example: ATD Complex

- 数据对象：
 $D = \{e1, e2 | e1, e2 \in R, R \text{ 为实数集}\}$
- 数据关系：
 $S = \{\langle e1, e2 \rangle | e1 \text{ 为实部}, e2 \text{ 为虚部}\}$
- 基本操作：
 Create(&C, x, y)
 GetReal(C)
 Add(C1, C2)

表示部分：

```
typedef struct{  
    float Realpart;  
    float Imagepart;  
} Complex;
```

实现部分：

```
void Create(&Complex c, float x, float y){  
    c.Realpart=x;  
    c.Imagepart=y;  
}  
float GetReal(Complex c){  
    return c.Realpart;  
}  
.....
```

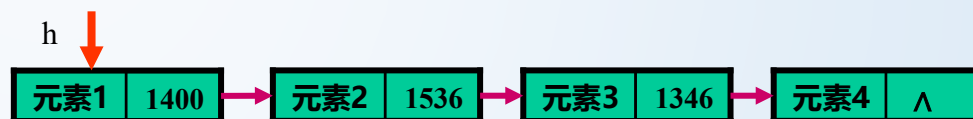
存储结构

- 顺序存储结构—借助元素在存储器中的相对位置来表示数据元素间的逻辑关系

存储地址	存储内容
L_0	元素1
L_0+m	元素2
.....	
$L_0+(i-1)*m$	元素i
.....	
$L_0+(n-1)*m$	元素n

$$\text{Loc}(\text{元素}i) = L_0 + (i-1)*m$$

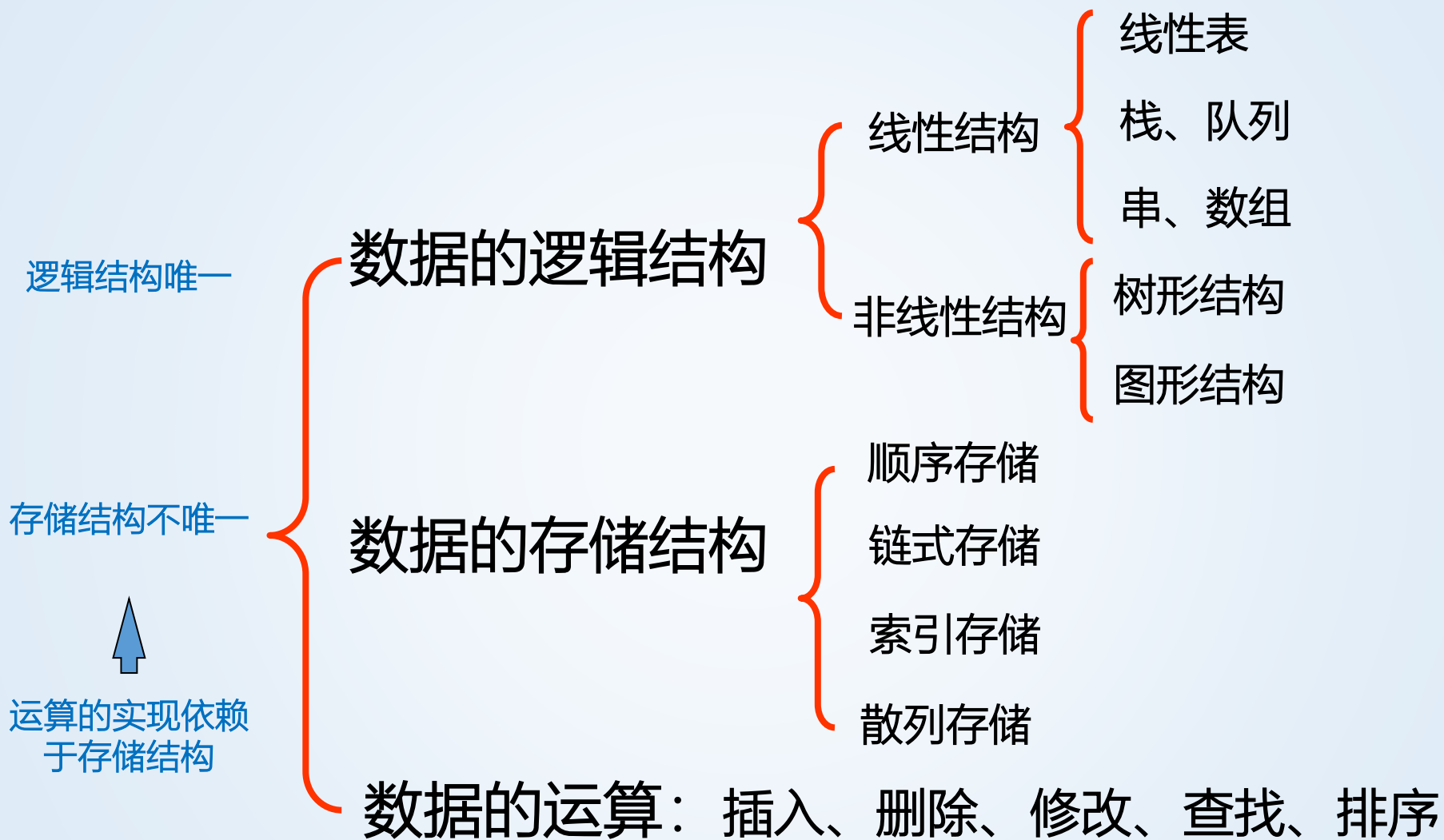
- 链式存储结构—借助指示元素存储地址的指针表示数据元素间的逻辑关系



1345

h

存储地址	存储内容	指针
1345	元素1	1400
1346	元素4	^
.....		
1400	元素2	1536
.....		
1536	元素3	1346





算法及算法分析

算法定义及特性, 算法时间复杂度,
空间复杂度分析

算法定义和特征

- 定义：

- 一个**有穷**的指令集，这些指令为解决某一特定任务规定了一个运算序列

- 特性：

- **输入** 有0个或多个输入
 - **输出** 有一个或多个输出(处理结果)
 - **确定性** 每步定义都是确切、无歧义的
 - **有穷性** 算法应在执行有穷步后结束
 - **有效性** 每一条运算应足够基本

算法描述-欧几里德算法

自然语言

- ①输入 m 和 n ;
- ②求 m 除以 n 的余数 r ;
- ③若 r 等于0, 则 n 为最大公约数, 算法结束; 否则执行 第④步;
- ④将 n 的值放在 m 中, 将 r 的值放在 n 中;
- ⑤重新执行第②步。

程序语言或伪代码

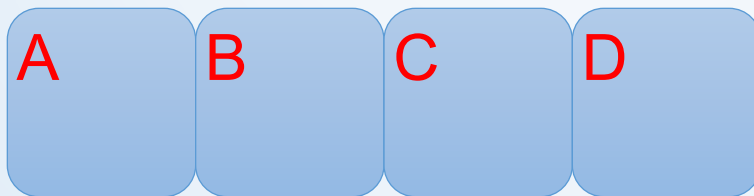
1. $r = m \% n$;
2. while($n \neq 0$){
 $m = n$;
 $n = r$;
 $r = m \% n$;}
3. output n

表达能力强, 抽象性强, 容易理解;

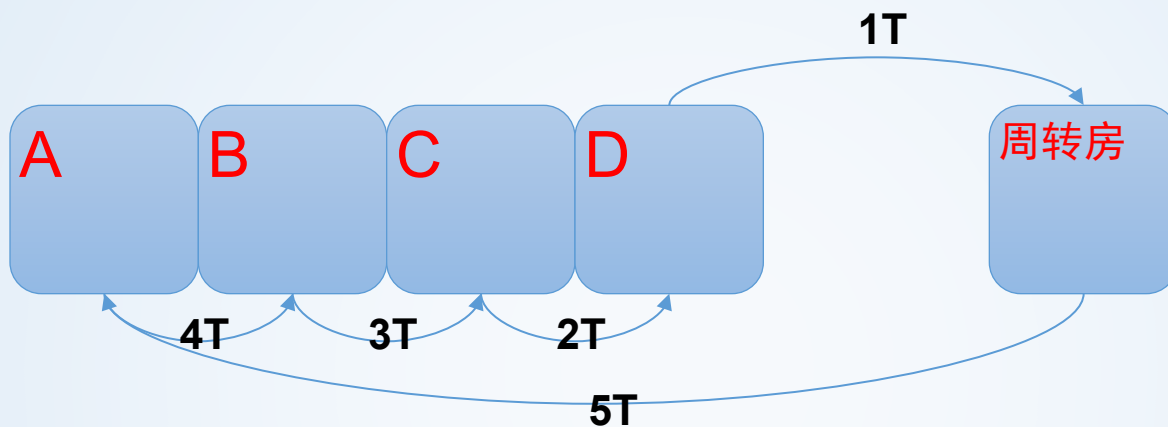
优点: 容易理解 缺点: 冗长、二义性

提出问题：如何评价算法？

- 假设有4间寝室（A/B/C/D），如果要将A寝室的物品搬到B寝室，B寝室的物品搬到C寝室，C寝室的物品搬到D寝室，D寝室的物品搬到A寝室，且规定只有当某间寝室为空时才能往进搬东西，物品从一间寝室搬到另一间寝室的时间为T。请描述完成上述任务的算法，并评价其算法优劣。

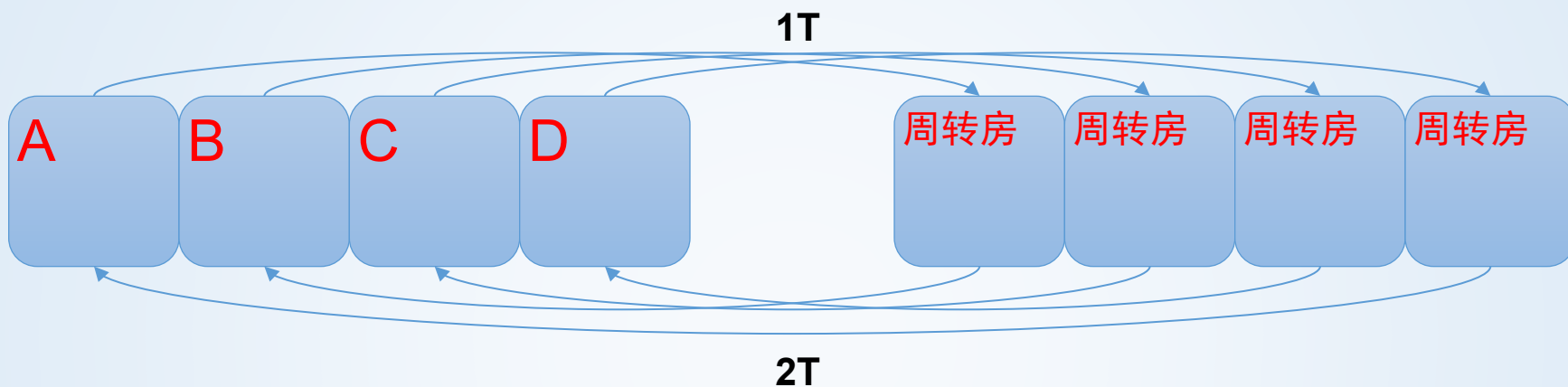


方案一：占用尽量少的额外空间



- 占用额外空间1
- 消耗时间5T

方案二：在最短时间内完成



- 占用额外空间4
- 消耗时间 $2T$

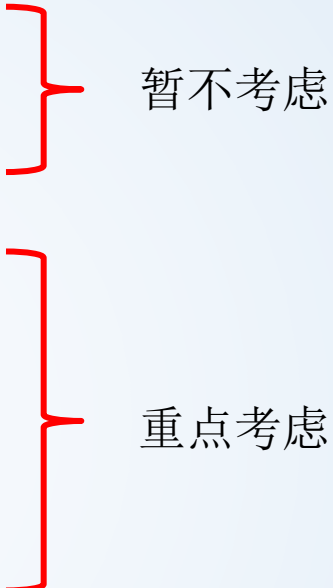
两个方案的区别

1. 辅助空间占用数量
2. 算力需求数量
3. 消耗时间

结论

- 评价程序性能
 - 分析或测量程序的时空开销
- 选择算法
 - 针对某一具体问题，在其不同解决方案之间进行空间和时间的权衡

影响程序性能评价的因素

- 计算机
 - 编译器及其选项
 - 问题规模（问题实例长度）
 - 具体输入
 - 代码本身（数据结构及算法）
- 
- 暂不考虑
- 重点考虑

空间复杂性

- 程序所需空间
 - 指令空间
 - 存储编译后的指令所需的空间
 - 与目标机器、编译器及选项有关
 - 不在本课程研究范围之内
 - 数据空间
 - 存储常量、简单变量、复合变量及动态分配的空间
 - 环境栈空间
 - 发生函数调用时所需的空间（递归调用和循环比较）

数据空间：变量所占空间

类型	占用字节数
char, unsigned char	1
short	2
int, unsigned int	4
long, unsigned long	4
float	4
double, long double	8
pointer	4 (64位系统及编译器长度为8)

- `double a[100];` → 800字节
- `int b[r][c];` → $4 * r * c$ 字节

程序空间分析

- 程序空间分为固定部分和可变部分
- 其中可变部分是指随问题规模变化的空间
 - 复合变量所需的空间
 - 动态分配的空间
 - 递归栈所需的空间

$$S(P) = c + S_p(\text{实例特征})$$

空间复杂性分析例1:累加

循环

```
template<class T>
T Sum(T a[], int n)
{
    T tsum=0;
    for(int i=0;i<n;i++)
        tsum+=a[i];
    return tsum;
}
```

消耗空间与n无关

$$S_{Sum}(n)=1$$

递归

```
template<class T>
T Rsum(T a[], int n)
{
    if(n>0)
        return Rsum(a,n-1)+a[n-1];
    return 0;
}
```

消耗空间(递归栈空间)与n有关

$$S_{Rsum}(n) = (\text{sizeof(a指针)} + \text{sizeof(int)} + \text{sizeof(函数指针)}) * n$$

$$S_{Sum}(n) < S_{Rsum}(n)$$

小结

- 一个程序的空间开销包括
 - 生成代码指令后所占的空间
 - 简单变量、常量所占的空间
 - 复合变量所占的空间
 - 动态分配的空间
 - 递归栈空间

重点考查
可变部分

时间复杂度分析

- 指标（计数对象）
- 渐近符号（ O 、 Ω 、 Θ 、 o ）
- 性能测量

时空分析对比

• 空间复杂性 $S(P) = c + S_p(\text{实例特征})$

• 时间复杂性 $T(P) = c + t_p(\text{实例特征})$

不变 可变
部分 部分

操作计数分析示例：多项式求值

$$P(x) = \sum_{i=0}^n c_i x^i = c_0 x^0 + c_1 x^1 + \dots + c_n x^n$$

鲁莽算法

```
template<class T>
T PolyEval(T coeff[], int n, const T& x)
{
    T y=1, value=coeff[0];
    for(int i=1;i<=n;i++)
    {
        y*=x;
        value+=y*coeff[i];
    }
    return value;
}
```

认定加法和乘法是关键操作，
该算法的关键操作有多少次呢？
对于问题规模n来说

累计将循环n次

} 每次循环做2个乘法和1个加法

备注：示例引自《数据结构、算法与应用》2.3节

多项式求值优化

$$P(x) = \sum_{i=0}^n c_i x^i = c_0 x^0 + c_1 x^1 + \dots + c_n x^n$$
$$= (\dots(((c_n * x + c_{n-1}) * x + c_{n-2}) * x + c_{n-3}) * x + \dots) * x + c_0$$

Horner算法

```
template<class T>
```

```
T Horner(T coeff[], int n, const T& x)
```

```
{
```

```
    T value=coeff[n];
```

```
    for(int i=1;i<=n;i++)
```

```
    {
```

```
        value = value * x + coeff[n-i];
```

```
    }
```

```
    return value;
```

```
}
```

对于问题规模n来说

累计将循环n次

} 每次循环做1个乘法和1个加法

**结论：Horner算法更快！
因为其关键操作数更少！**

执行步数

- 操作计数
 - 优点：抓住主要矛盾（程序的主要开销）
 - 缺点：忽略一些所谓次要操作，造成不够准确
- 执行步数
 - 程序的实际工作量，与程序步数（指定的语法片段）有关
 - 涵盖更多操作/语句
 - 也与问题规模有关
 - 一般做法是在有意义的语法片段旁加count变量

执行步数分析示例1：一般程序

```
template<class T>
T Sum(T a[], int n)
{
    T tsum = 0;
    count++; // tsum 初始化
    int i;
    for (i=0; i < n; i++) {
        tsum += a[i];
        count++; // 累加并赋值
    }
    count+=2*n; // 循环变量赋值及判断

    count++; // for return
    return tsum;
}
```

备注：示例引自《数据结构、算法与应用》

函数Sum的执行步数

语句	s/e	频率	总步数
T Sum(T a[], int n)	0	0	0
{	0	0	0
T tsum = 0;	1	1	1
for (int i = 0; i < n; i++)	2	$n + 1$	$2n + 2$
tsum += a[i];	1	n	n
return tsum;	1	1	1
}	0	0	0
总计			$3n + 4$

执行步数分析示例2：递归程序

```
template<class T>
```

```
T Rsum(T a[], int n)
```

```
{
```

```
    count++; // for if conditional
```

```
    if (n > 0) {count++; count++;
```

```
        // for return and Rsum invocation
```

```
        return Rsum(a, n-1) + a[n-1];}
```

```
    count++; // for return
```

```
    return 0;
```

```
}
```

$$t_{\text{Rsum}}(0)=2, \quad t_{\text{Rsum}}(n)=3+t_{\text{Rsum}}(n-1)$$

→ $t_{\text{Rsum}}(n)=3n+2$

尾调用；尾递归

函数Rsum的执行步数

语句	s/e	频率	总步数
T Rsum(T a[], int n)	0	0	0
{	0	0	0
if (n > 0)	1	$n + 1$	$n + 1$
return Rsum(a, n-1) + a[n-1];	2	n	$2n$
return 0;	1	1	1
}	0	0	0
总计			$3n + 2$

矩阵转置函数的执行步数

语句	s/e	频率	总步数
<code>void Transpose(T **a, int rows)</code>	0	0	0
<code>{</code>	0	0	0
<code> for (int i = 0; i < rows; i++)</code>	2	$rows + 1$	$2(rows + 1)$
<code> for (int j = i+1; j < rows; j++)</code>	2	$rows*(rows+1)/2$	$rows*(rows+1)$
<code> Swap(a[i][j], a[j][i]);</code>	1	$rows*(rows-1)/2$	$rows*(rows-1)/2$
<code>}</code>	0	0	0
总计			$3/2*rows^2+5/2*rows+2$

$$\sum_{i=0}^{rows-1} (rows - i) = \sum_{i=1}^{rows} i = rows(rows + 1) / 2$$

$$\sum_{i=0}^{rows-1} (rows - i - 1) = \sum_{i=0}^{rows-1} i = rows(rows - 1) / 2$$



时间复杂度分析

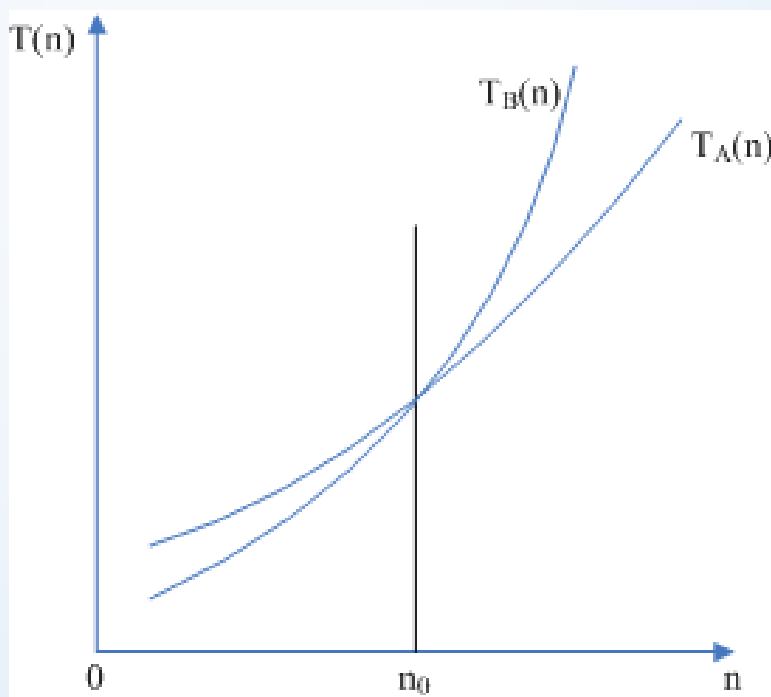
渐近符号表示法 (asymptotic notation)

问题提出

- 操作计数和执行步数的作用？
 - 比较两个功能相同的程序的时间复杂性【横向】
 - 预测随实例规模的变化，程序运行时间的变化【纵向】
- 执行步试图比“关键操作”更精确
- “更精确”是没有必要的
 - 两个程序时间复杂性： $c_1n^2+c_2n$ 和 c_3n
 - 总存在一个点，当 n 超过此值，后者更快
 - “渐进”：大实例特征（趋向无穷）下，程序时间复杂性函数的“变化”情况

复杂度的渐进性质

- 如果解决问题P的程序A和程序B，其时间复杂度分别是 $T_A(n)$ 和 $T_B(n)$ ，则判断A、B性能优劣的标准是查看在 n 足够大时 $T_A(n)$ 和 $T_B(n)$ 的大小关系



大写O符号 Big-Oh

- 函数上界

- 定义：

$f(n)=O(g(n))$ ，当且仅当存在正常数 c 和 n_0 ，使得对所有 $n \geq n_0$ ，有 $f(n) \leq cg(n)$

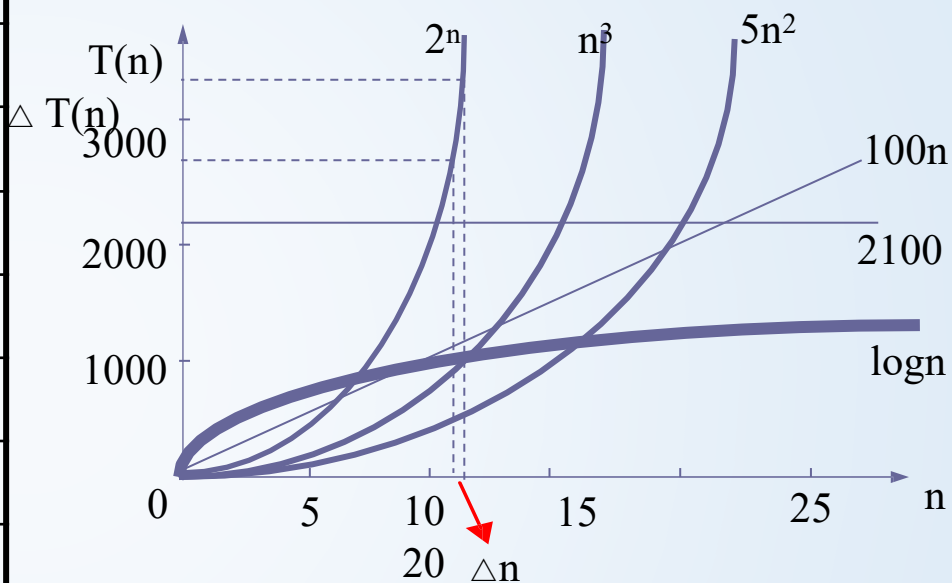
- f 至多是 g 的 c 倍，对足够大的 n ， g 是 f 的上界（不考虑常数因子 c ）
- g 取简单函数——容易研究 f 的上界
- 注意：Big-Oh不一定是最小上界。但是通常会求最小上界

常用做g的简单函数

快、简单

函数	名称
1	常数
$\log n$	对数
n	线性
$n \log n$	n 个 $\log n$
n^2	平方
n^3	立方
2^n	指数
$n!$	阶乘

慢、复杂



对数没有给出对数基，因为

$\log_a n = \log_b n / \log_b a$ ，仅常数不同，相差 $\log_b a$ 倍

大O符号例子(1)

$f(n) = O(g(n))$, 当且仅当
存在正常数 c 和 n_0 , 使得对
所有 $n \geq n_0$, 有 $f(n) \leq cg(n)$

- 线性函数
- $f(n) = 3n + 2$
 - $n \geq 2$ 时, $3n + 2 \leq 3n + n = 4n$, $f(n) = O(n)$ //此为最小上界
 - $n > 0$ 时, $n \geq 1$, $3n + 2 \leq 3n + 2n = 5n$
- $f(n) = 3n + 3$: $n \geq 3$ 时, $3n + 3 \leq 3n + n = 4n$
- $f(n) = 100n + 6$: $n \geq 6$ 时, $100n + 6 \leq 100n + n = 101n$

c 和 n_0 并不重要

大O符号例子 (2)

$f(n) = O(g(n))$, 当且仅当
存在正常数 c 和 n_0 , 使得对
所有 $n \geq n_0$, 有 $f(n) \leq cg(n)$

- 平方函数

- $f(n) = 10n^2 + 4n + 2$

- $n \geq 2$ 时, $f(n) \leq 10n^2 + 5n$

- 当 $n \geq 5$ 时, $5n \leq n^2$, 因此 $n \geq n_0 = 5$ 时,

- $f(n) \leq 10n^2 + n^2 = 11n^2$, 所以 $f(n) = O(n^2)$

- $f(n) = 1000n^2 + 100n - 6$

- 对于所有 n 有 $f(n) \leq 1000n^2 + 100n$,

- 对于 $n \geq 100$, 有 $100n \leq n^2$,

- 因此对于 $n \geq n_0 = 100$, 有 $f(n) < 1001n^2$,

- 所以 $f(n) = O(n^2)$ //此为最小上界

大O符号例子 (3)

$f(n) = O(g(n))$, 当且仅当
存在正常数 c 和 n_0 , 使得对
所有 $n \geq n_0$, 有 $f(n) \leq cg(n)$

- 指数函数

- $f(n) = 6 \cdot 2^n + n^2$

- $n \geq 4$ 时, $n^2 \leq 2^n$,

- 所以对于 $n \geq 4$, $f(n) \leq 6 \cdot 2^n + 2^n = 7 \cdot 2^n$,

- 因此 $6 \cdot 2^n + n^2 = O(2^n)$

- 常数函数

- $f(n) = 9$ 或 $f(n) = 2033$, $f(n) = O(1)$, 证明很简单:

- $f(n) = 9 \leq 9 \cdot 1$, $c = 9$, $n_0 = 0$

- $f(n) = 2033 \leq 2033 \cdot 1$, $c = 2033$, $n_0 = 0$

大O符号例子（4）

- 松散界限

- 当 $n \geq 2$ 时, $3n+3 \leq 3n^2 \rightarrow 3n+3 = O(n^2)$, 不是最小上界
- 当 $n \geq 2$ 时, $10n^2+4n+2 \leq 10n^4 \rightarrow 10n^2+4n+2 = O(n^4)$
- $6n2^n+20 = O(n^22^n)$, 更小上界 $n2^n$
- 逐步用更低阶的函数替换高阶函数, 直到找到最小上界
- 高阶函数一定是低阶函数的松散上界。

错误界限

- $3n+2 \neq O(1)$, 反证法:
 - 假定存在 c 及 n_0 , 使得对所有的 $n \geq n_0$, 有 $3n+2 \leq c \rightarrow n \leq (c-2)/3$, 因此取 $n > \max\{n_0, (c-2)/3\}$, 矛盾!
- $10n^2+4n+2 \neq O(n)$
 - 假定存在 c 和 n_0 , 使得对于所有的 $n \geq n_0$, 有 $10n^2+4n+2 \leq cn \rightarrow 10n+4+2/n \leq c$ 取 $n > \max\{n_0, (c-4)/10\}$, 矛盾

多项式的阶

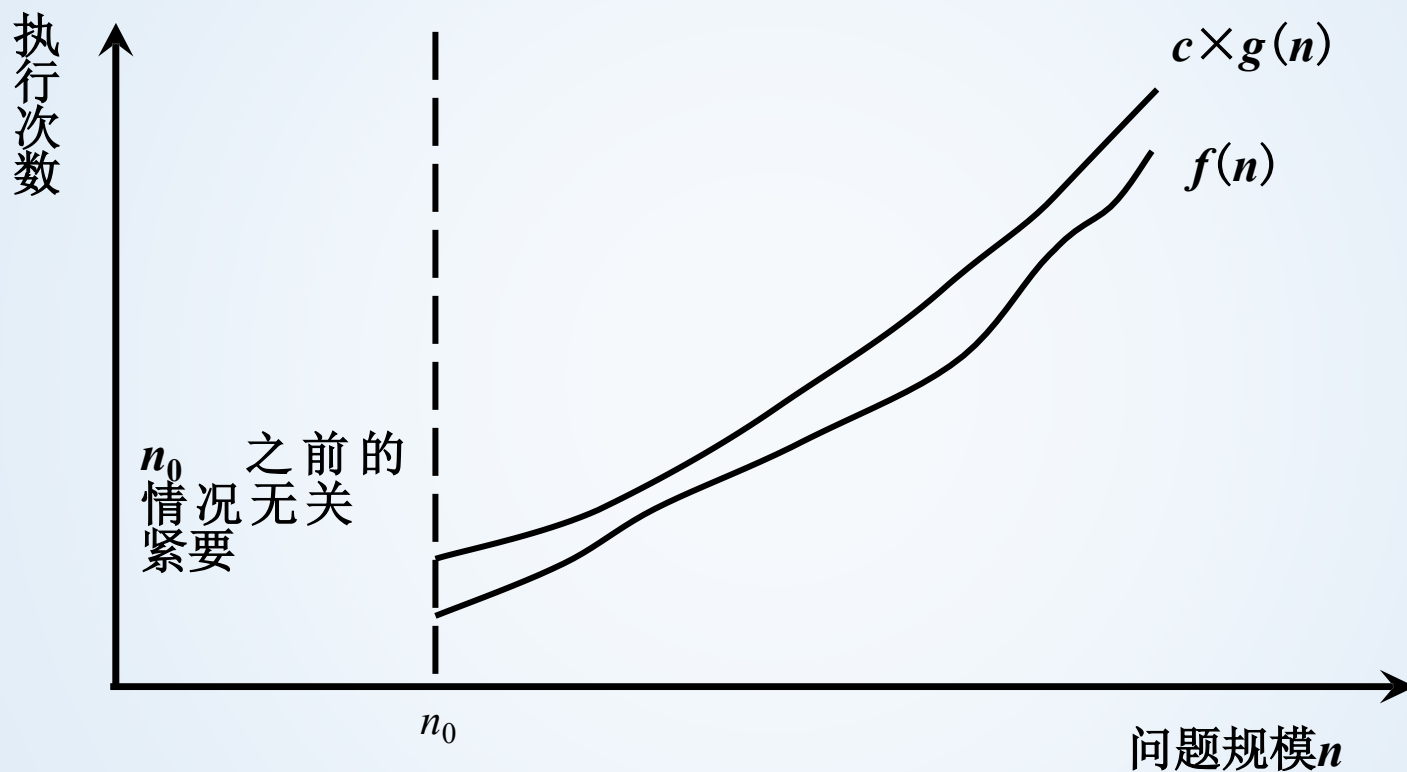
• 定理1: 如果 $f(n)=a_m n^m+\dots+a_1 n+a_0$ 且 $a_m>0$, 则 $f(n)=O(n^m)$

证明:

对所有 $n \geq 1$

$$\begin{aligned} f(n) &\leq \sum_{i=0}^m |a_i| n^i \\ &\leq n^m \sum_{i=0}^m |a_i| n^{i-m} \\ &\leq n^m \sum_{i=0}^m |a_i| \end{aligned}$$

大O原理图示



当问题规模充分大时在渐近意义下的阶

小结

- 关于大O符号有如下认识
 - 时间复杂度的“**级别**”比“具体量”更重要！
 - 确定时间消耗是什么级别，而非具体多少
 - 是问题规模的函数
 - 根据渐进性质，考虑问题足够大的情况
 - 本质上是最差情况，这一点符合工业界需求

Ω 符号(Big-Omega)

- 函数下界

- 定义:

$f(n) = \Omega(g(n))$, 当且仅当存在正常数 c 和 n_0 , 使得对所有 $n \geq n_0$, 有 $f(n) \geq c g(n)$

- f 至少是 g 的 c 倍, 对足够大的 n , g 是 f 的一个下界

Ω 符号例子 (1)

- 线性函数

- 对于所有的 n , 有 $f(n)=3n+2>3n \rightarrow f(n)=\Omega(n)$
- $f(n)=3n+3>3n \rightarrow f(n)=\Omega(n)$
- $f(n)=100n+6>100n$, 所以 $100n+6=\Omega(n)$

- 平方函数

- 对所有 $n \geq 0$, $f(n)=10n^2+4n+2>10n^2 \rightarrow f(n)=\Omega(n^2)$
- $1000n^2+100n-6=\Omega(n^2)$

- 指数函数

- $6 \cdot 2^n + n^2 > 6 \cdot 2^n \rightarrow 6 \cdot 2^n + n^2 = \Omega(2^n)$

Ω 符号例子 (2)

- 非最大下界

- $3n+3=\Omega(1)$, $10n^2+4n+2=\Omega(n)$,
 $10n^2+4n+2=\Omega(1)$, $6*2^n+n^2=\Omega(n^{100})$

- $6*2^n+n^2=\Omega(n^{50.2})$, $6*2^n+n^2=\Omega(n^2)$,
 $6*2^n+n^2=\Omega(n)$ 和 $6*2^n+n^2=\Omega(1)$

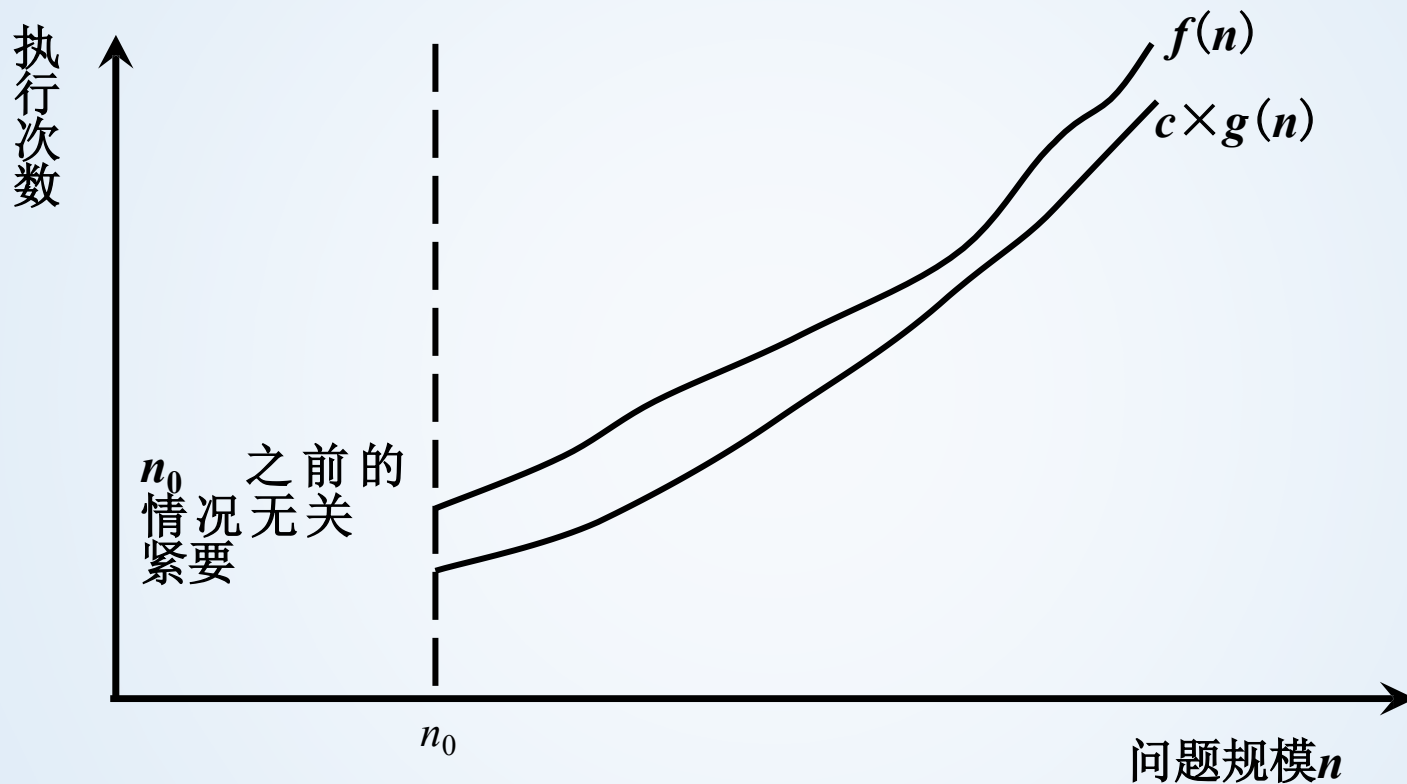
- $3n+2 \neq \Omega(n^2)$

- 假定 $3n+2=\Omega(n^2) \rightarrow$ 存在正数 c 和 n_0 , 使得对所有 $n \geq n_0$, 有
 $3n+2 \geq cn^2 \rightarrow cn^2/(3n+2) \leq 1$, 左边随 n 的增大而变得无限大, 右边
常数,
不等式变为不成立

关于 Ω 的多项式定理

- 定理2: 如果 $f(n)=a_m n^m+\dots+a_1 n+a_0$ 且 $a_m>0$, 则 $f(n)=\Omega(n^m)$
- 例: $3n+2=\Omega(n)$, $10n^2+4n+2=\Omega(n^2)$,
 $100n^4+3500n^2+82n+8=\Omega(n^4)$

Ω 原理图示



当问题规模充分大时在渐近意义下的阶

Θ 符号(Big-Theta)

- 同一个 g 既作为 f 的上界，又作为下界

- 定义：

$f(n)=\Theta(g(n))$ ，当且仅当存在正常数 c_1 、 c_2 和 n_0 ，使得对所有 $n \geq n_0$ ，有

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$

- f 介于 g 的 c_1 倍和 c_2 倍之间，对足够大的 n ， g 既是 f 的上界也是下界
- 也就是当 O 与 Ω 相同时，该函数为 f 函数的 Θ

Θ 符号例子

- 从前例可知: $3n+2=\Theta(n)$, $3n+3=\Theta(n)$, $100n+6=\Theta(n)$,
 $10n^2+4n+2=\Theta(n^2)$, $1000n^2+100n-6=\Theta(n^2)$,
 $6*2^n+n^2=\Theta(2^n)$
- 对 $n \geq 16$, $\log_2 n < 10*\log_2 n + 4 \leq 11*\log_2 n \rightarrow 10*\log_2 n + 4 = \Theta(\log n)$
- $3n+2 \neq \Theta(1)$
- $3n+3 \neq \Theta(n^2)$
- $10n^2+4n+2 \neq \Theta(n)$
- $6*2^n+n^2 \neq \Theta(n^2)$

关于 Θ 的多项式定理

- 定理3: 如果 $f(n)=a_m n^m+\dots+a_1 n+a_0$ 且 $a_m>0$, 则 $f(n)=\Theta(n^m)$
- 例: $3n+2=\Theta(n)$, $10n^2+4n+2=\Theta(n^2)$,
 $100n^4+3500n^2+82n+8=\Theta(n^4)$

小写o符号(little-Oh)

- 定义:

$f(n)=o(g(n))$, 当且仅当 $f(n)=O(g(n))$, 且 $f(n)\neq\Omega(g(n))$

- 例如:

- $3n+2=O(n^2)$ 且 $3n+2\neq\Omega(n^2)\rightarrow 3n+2=o(n^2)$ 但 $3n+2\neq o(n)$

- $10n^2+4n+2=o(n^3)$, 但 $10n^2+4n+2\neq o(n^2)$



算法分析

最好情况、最坏情况、平均情况

算法分析

- 例：在一维整型数组A[n]中顺序查找与给定值k相等的元素（假设该数组中有且仅有一个元素值为k）。

```
int Find(int A[ ], int n)
{
    for (i=0; i<n; i++)
        if (A[i] == k) break;
    return i;
}
```

- 基本语句的执行次数是否只和问题规模有关？
结论：如果问题规模相同，时间代价与输入数据（的分布）有关，则需要分析最好情况（ $O(1)$ ）、最坏情况($O(n)$)、平均情况($O(n)$)

示例：求最大子序列 Given (possibly negative) integers $A_1, A_2, \dots, A_N$, find the maximum value of $\sum_{k=i}^j A_k$.

Algorithm 1

```
int MaxSubsequenceSum ( const int A[ ], int N )
{
    int ThisSum, MaxSum, i, j, k;
    /* 1*/ MaxSum = 0; /* initialize the maximum sum */
    /* 2*/ for( i = 0; i < N; i++ ) /* start from A[ i ] */
    /* 3*/     for( j = i; j < N; j++ ) { /* end at A[ j ] */
    /* 4*/         ThisSum = 0;
    /* 5*/         for( k = i; k <= j; k++ )
    /* 6*/             ThisSum += A[ k ]; /* sum from A[ i ] to A[ j ] */
    /* 7*/         if ( ThisSum > MaxSum )
    /* 8*/             MaxSum = ThisSum; /* update max sum */
    /* 9*/     } /* end for-j and for-i */
    return MaxSum;
}
```

$T(N) = O(N^3)$ (Detailed analysis is given on p.18-19.)

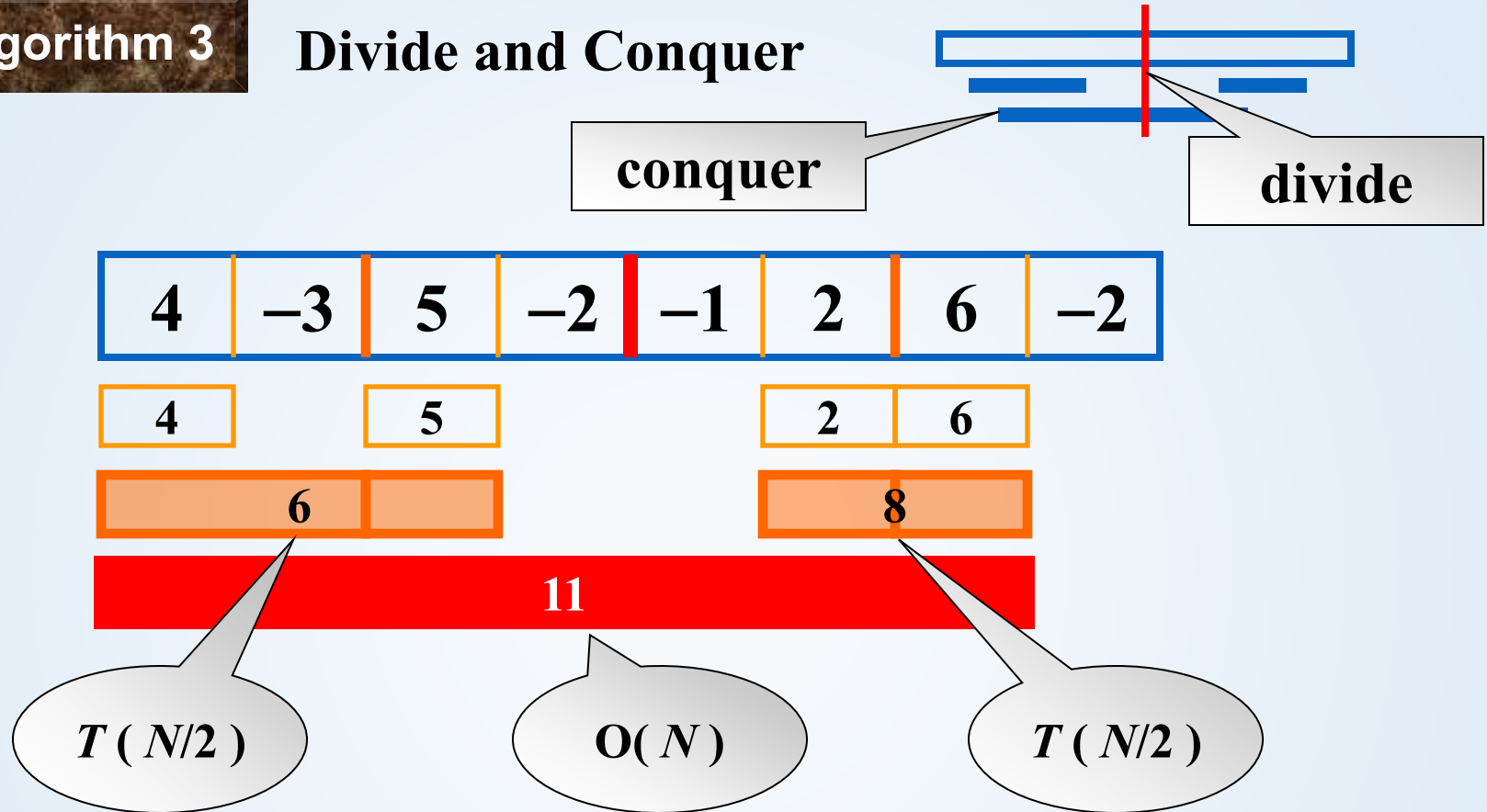
Algorithm 2

```
int MaxSubsequenceSum ( const int A[ ], int N )
{
    int ThisSum, MaxSum, i, j;
    /* 1*/ MaxSum = 0; /* initialize the maximum sum */
    /* 2*/ for( i = 0; i < N; i++ ) { /* start from A[ i ] */
    /* 3*/     ThisSum = 0;
    /* 4*/     for( j = i; j < N; j++ ) { /* end at A[ j ] */
    /* 5*/         ThisSum += A[ j ]; /* sum from A[ i ] to A[ j ] */
    /* 6*/         if ( ThisSum > MaxSum )
    /* 7*/             MaxSum = ThisSum; /* update max sum */
        } /* end for-j */
    } /* end for-i */
    /* 8*/ return MaxSum;
}
```

$$T(N) = O(N^2)$$

Algorithm 3

Divide and Conquer



$$T(N) = 2T(N/2) + cN, \quad T(1) = O(1)$$

$$= 2[2T(N/2^2) + cN/2] + cN$$

$$= 2^k O(1) + ckN \quad \text{where } N/2^k$$

$$= O(N \log N)$$

Also true for
 $N \neq 2^k$

Algorithm 4

On-line Algorithm

```
int MaxSubsequenceSum( const int A[ ], int N )
{
    int ThisSum, MaxSum, j;
/* 1*/   ThisSum = MaxSum = 0;
/* 2*/   for ( j = 0; j < N; j++ ) {
/* 3*/       ThisSum += A[ j ];
/* 4*/       if ( ThisSum > MaxSum )
/* 5*/           MaxSum = ThisSum;
/* 6*/       else if ( ThisSum < 0 )
/* 7*/           ThisSum = 0;
    } /* end for-j */
/* 8*/   return MaxSum;
}
```



At any point in time, the algorithm can correctly give an answer to the **subsequence** problem for the data it has already read.

$$T(N) = O(N)$$

A[] is scanned **once** only.

Running times of several algorithms for maximum subsequence sum (in seconds)

Algorithm		1	2	3	4
Time		$O(N^3)$	$O(N^2)$	$O(N \log N)$	$O(N)$
Input Size	$N=10$	0.00103	0.00045	0.00066	0.00034
	$N=100$	0.47015	0.01112	0.00486	0.00063
	$N=1,000$	448.77	1.1233	0.05843	0.00333
	$N=10,000$	NA	111.13	0.68631	0.03042
	$N=100,000$	NA	NA	8.0113	0.29832

Note: The time required to read the input is not included.



性能测量方法

Performance measurement

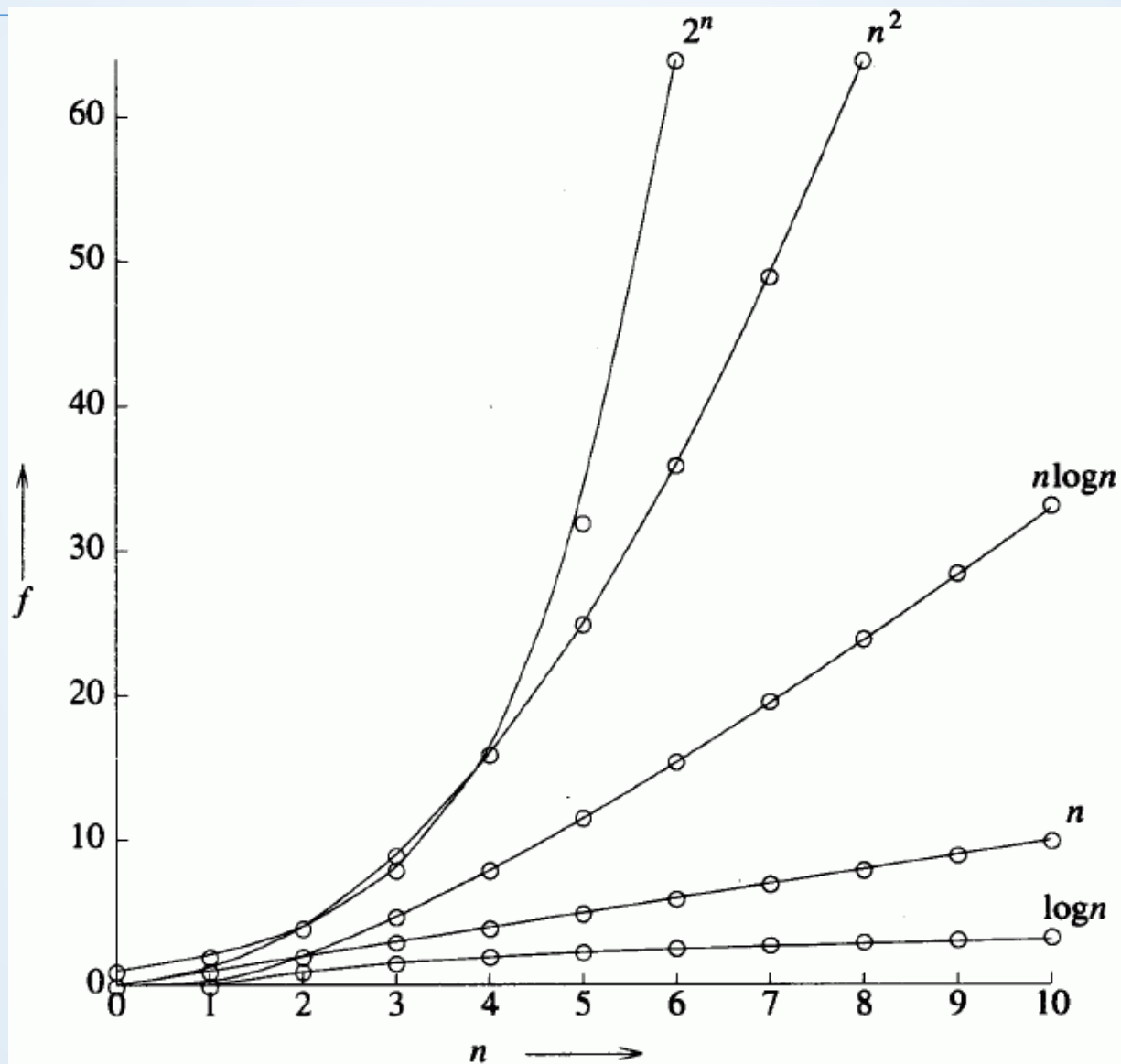
实际复杂性

- 利用渐进复杂性
 - 比较：两个解决相同问题的程序，其时间随问题规模变化而变化情况
- $P: \Theta(n)$, $Q: \Theta(n^2) \rightarrow n$ 足够大, P 快
- “足够大” —— 应考虑问题实际规模,
 $10^6 n$ 实际中几乎总是比 n^2 慢

各种函数的渐进变化表

$\log n$	n	$n \log n$	n^2	n^3	2^n
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4 096	65 536
5	32	160	1 024	32 768	4 294 967 296

各种函数的渐进曲线图



实际性能测量

- C和C++计时函数

- 示例如下：

```
//#include <windows.h>
LARGE_INTEGER t1, t2, tc;
QueryPerformanceFrequency(&tc);
QueryPerformanceCounter(&t1);
fastSquareMatrixMultiply(a,b,c,5000);
QueryPerformanceCounter(&t2);
cout << " time5000:" <<
(t2.QuadPart - t1.QuadPart)*1.0 / tc.QuadPart
<< endl;
```

序号	函数	类型	精度级别	时间
1	time	C系统调用	低	<1s
2	clock	C系统调用	低	<10ms
3	timeGetTime	Windows API	中	<1ms
4	QueryPerformanceCounter	Windows API	高	<0.1ms
5	GetTickCount	Windows API	中	<1ms
6	RDTSC	指令	高	<0.1ms
7	gettimeofday	linux环境下C系统调用	高	<0.1ms

缓存对算法性能的影响

- 通常算法分析针对内存足够大的理想情况
- 如果内存有限则需要对不同级别缓存进行读取，性能影响较大
- 示例：矩阵乘法

```
void fastSquareMatrixMultiply(int ** a, int ** b, int ** c, int n)
```

```
{  
    for (int i = 0; i < n; i++)  
        for (int j = 0; j < n; j++)  
            c[i][j] = 0;  
  
    for (int i = 0; i < n; i++)  
        for (int j = 0; j < n; j++)  
            for (int k = 0; k < n; k++)  
  
                c[i][j] += a[i][k] * b[k][j];  
}
```

n	ijk顺序	ikj顺序
500	0.859844	0.500231
1000	6.89394	4.04951
2000	77.4072	33.5267
5000		487

运行环境：
64位win10
CPU i7 6600u(2.6GHz),
RAM 8G,I1 128K,I2 512K,I3 4.0M

备注：示例引自《数据结构、算法与应用》第4.5.3节

总结

- 评判程序性能的两个标准
 - 空间复杂性
 - 时间复杂性
 - 分析法
 - 实验法
- 思考
 - 如果在写完某个程序后发现运行特别慢，必须加以改进，该如何做？

The End!